JAVA PROGRAMMING / DSA

# Chapter 36: Tries (Prefix Trees)

*Personal Notes*

Some problems are all about PREFIXES. Autocomplete suggests words that start with what you typed. Spellchecker flags words that don't exist in a dictionary. IP routing matches the longest prefix. All of these need a data structure that can answer "does anything start with this?" almost instantly.

That data structure is the TRIE (rhymes with "try"), also called a PREFIX TREE. It's a specialized tree that stores strings by their characters, sharing common prefixes to save space and enabling O(length) lookups. It's a small topic but shows up in a lot of Google-style interview problems.

## 1. What is a Trie?

A Trie is a tree-like data structure where each NODE represents a CHARACTER, and each PATH from the root to a node forms a PREFIX. Words are stored implicitly along paths — you don't store the whole string at any single node.

### Example: Trie Containing "cat", "car", "cart", "dog"

```
                    (root)
                   /      \
                  c        d
                  |        |
                  a        o
                 / \       |
                t   r      g*
               *   *
                   |
                   t*

    Each letter is one node.
    * marks the end of a valid word.
    "car" and "cart" share the prefix "car" — the 't' is a child of 'r'.
```

### Key Terms

| Term | Meaning |
|---|---|
| **Root** | Empty top node — no character |
| **Node** | Represents ONE character in a word |
| **Path from root** | Spells out a prefix |
| **End-of-word marker** | Boolean flag saying "this node completes a valid word" |

| Term | Meaning |
|---|---|
| **Children** | A map or array pointing to next characters |
| **Depth** | Same as length of the prefix ending at that node |

## 2. Why Not Just Use a HashSet?

HashSet gives O(1) EXACT lookup — but it can't answer prefix questions efficiently.

| Operation | HashSet | Trie |
|---|---|---|
| **Exact word lookup** | O(1) average | O(length of word) |
| **Does anything start with X?** | O(n) — scan every word | O(length of X) |
| **List all words starting with X** | O(n × avg length) | O(matches) after prefix traversal |
| **Space (many common prefixes)** | Stores full duplicate strings | Shares prefixes — often smaller |
| **Ordered iteration** | No | Yes — DFS gives sorted order |

> **The trie's advantage:** For EXACT lookup, HashSet wins slightly. But the moment prefix matching comes up ("words starting with 'app'"), HashSet has to scan everything. A trie navigates directly to the matching subtree in O(prefix length). That's the reason autocomplete, spellcheck, and dictionary systems all use tries under the hood.

## 3. Building a Trie in Java

The standard Java implementation uses a TrieNode class holding a Map (or fixed array for lowercase-only alphabets) of children and a boolean end-of-word flag.

### The TrieNode Class

```
class TrieNode {
    Map<Character, TrieNode> children = new HashMap<>();
    boolean isEnd = false;
}
```

### Alternative: Fixed-Size Array (Faster for Lowercase-Only)

```
class TrieNode {
    TrieNode[] children = new TrieNode[26];   // for a-z only
    boolean isEnd = false;
}
```

```
// To get child for character c:
int index = c - 'a';
TrieNode child = children[index];
```

> **Map vs Array:** HashMap is more flexible (handles any character) but has hash overhead. int[26] is faster for problems limited to lowercase letters — no hashing, direct index. In interviews, ask about the character set — if lowercase-only, use the array version for speed. For general text, use HashMap.

# 4. Core Operations

## 4.1 Insert a Word

```
public void insert(String word) {
    TrieNode curr = root;
    for (char c : word.toCharArray()) {
        curr.children.putIfAbsent(c, new TrieNode());
        curr = curr.children.get(c);
    }
    curr.isEnd = true;
}
```

Walk one character at a time. If the child doesn't exist, create it. At the end, mark the last node as end-of-word.

## 4.2 Search for a Word

```
public boolean search(String word) {
    TrieNode node = findNode(word);
    return node != null && node.isEnd;
}

private TrieNode findNode(String prefix) {
    TrieNode curr = root;
    for (char c : prefix.toCharArray()) {
        if (!curr.children.containsKey(c)) return null;
        curr = curr.children.get(c);
    }
    return curr;
}
```

Search returns true ONLY if we reach a node AND that node's isEnd is true. Otherwise we just followed a prefix but not a complete word.

## 4.3 Starts With (Prefix Match)

```
public boolean startsWith(String prefix) {
    return findNode(prefix) != null;
}
```

Same walk as search — but we don't care whether the last node is isEnd. Just being able to reach that node means SOMETHING has this prefix.

### Full Trie Class

```
class Trie {
    private TrieNode root = new TrieNode();

    public void insert(String word) {
        TrieNode curr = root;
        for (char c : word.toCharArray()) {
            curr.children.putIfAbsent(c, new TrieNode());
            curr = curr.children.get(c);
        }
        curr.isEnd = true;
    }

    public boolean search(String word) {
        TrieNode n = findNode(word);
        return n != null && n.isEnd;
    }

    public boolean startsWith(String prefix) {
        return findNode(prefix) != null;
    }

    private TrieNode findNode(String s) {
        TrieNode curr = root;
        for (char c : s.toCharArray()) {
            if (!curr.children.containsKey(c)) return null;
            curr = curr.children.get(c);
        }
        return curr;
    }
}
```

## 5. Walking Through Insert and Search

### Insert "cat" then "car"

```
 Insert "cat":
    root → c → a → t*    (* = isEnd)

 Insert "car":
    root → c → a → t*
                  \
                   r*      (existing prefix "ca" is REUSED)

 Only ONE new node (r) is created — the "ca" prefix is shared.
```

### Search "cat"

```
Start at root
Look for 'c'  ✓  move to c-node
Look for 'a'  ✓  move to a-node
Look for 't'  ✓  move to t-node
t-node.isEnd  ✓  → return true
```

### Search "ca" (a prefix but not a word)

```
Start at root
Look for 'c'  ✓
Look for 'a'  ✓  reached a-node
a-node.isEnd  ✗  → return false

But startsWith("ca") would return true — we successfully walked "ca".
```

## 6. Deleting a Word (Optional but Useful)

Deletion is trickier than insertion — you can't just remove nodes, because other words might share the prefix. The standard approach uses recursion, unsetting isEnd and cleaning up only when truly safe.

```
public void delete(String word) {
    delete(root, word, 0);
}

private boolean delete(TrieNode curr, String word, int i) {
    if (i == word.length()) {
        if (!curr.isEnd) return false;      // word not present
        curr.isEnd = false;
        return curr.children.isEmpty();     // true = safe to delete this
node
    }

    char c = word.charAt(i);
    TrieNode child = curr.children.get(c);
    if (child == null) return false;

    boolean shouldDeleteChild = delete(child, word, i + 1);

    if (shouldDeleteChild) {
        curr.children.remove(c);
        return !curr.isEnd && curr.children.isEmpty();
    }
    return false;
}
```

**Delete is easy to get wrong:** Two common bugs — deleting nodes that belong to another word, or leaving orphan nodes behind. The recursive approach above returns "should the caller delete me?"

only when it's safe (no children AND not marking any other word). Draw a picture with 2-3 overlapping words to verify.

## 7. Complexity Analysis

| Operation | Time | Space (per word) |
|---|---|---|
| **Insert** | O(L) | O(L × alphabet) |
| **Search** | O(L) | - |
| **StartsWith** | O(L) | - |
| **Delete** | O(L) | - |

L = length of the word or prefix. Note that time is INDEPENDENT of how many words are in the trie — you only walk down the path of THIS word. That's the trie's key advantage.

**Total space consideration:** For N words of average length L, worst-case space is O(N × L × alphabet) — but common prefixes dramatically reduce this in practice. For a dictionary of 100K English words, tries typically use similar space to storing all words in an array, but give you prefix search for free.

## 8. Classic Problem 1 — Implement a Trie

The most famous trie interview question is to just implement one. LeetCode #208 asks for exactly this — a class with insert, search, and startsWith. If you can do the code from Section 4 by heart, you're set.

The code we've already written IS the answer. Practice writing it without looking — that's often enough to pass the question.

## 9. Classic Problem 2 — Autocomplete / Word Suggestions

Given a prefix, return all words in the trie that start with that prefix. Two-step algorithm: (1) navigate to the prefix node, (2) DFS from there collecting all valid words.

```
public List<String> autocomplete(String prefix) {
    List<String> results = new ArrayList<>();
    TrieNode node = findNode(prefix);
    if (node == null) return results;
    dfs(node, prefix, results);
    return results;
}

private void dfs(TrieNode node, String current, List<String> results) {
    if (node.isEnd) results.add(current);
```

```
    for (var entry : node.children.entrySet()) {
        dfs(entry.getValue(), current + entry.getKey(), results);
    }
}
```

### Example

```
 Trie contains: apple, apply, apricot, banana

 autocomplete("app"):
    Navigate to "app" node.
    DFS collects: "apple", "apply"
    Return: ["apple", "apply"]

 autocomplete("ap"):
    Navigate to "ap" node.
    DFS collects: "apple", "apply", "apricot"
    Return: ["apple", "apply", "apricot"]

 autocomplete("z"):
    No 'z' child at root. Return: []
```

## 10. Classic Problem 3 — Word Search II

Given a 2D board of letters and a list of words, return all words that can be formed by traversing adjacent cells. Solving this WITHOUT a trie would run DFS separately for each word — slow. WITH a trie, one DFS pass can find all words at once.

### The Trick

- Insert all words into a trie
- DFS from each cell of the board — but only follow paths that ALSO exist in the trie
- When your trie walk reaches a node with isEnd = true, you've found a word

```
class Solution {
    List<String> result = new ArrayList<>();

    public List<String> findWords(char[][] board, String[] words) {
        Trie trie = new Trie();
        for (String w : words) trie.insert(w);

        for (int r = 0; r < board.length; r++)
            for (int c = 0; c < board[0].length; c++)
                dfs(board, r, c, trie.root, new StringBuilder());

        return result;
    }
```

```
    void dfs(char[][] board, int r, int c, TrieNode node, StringBuilder
path) {
        if (r < 0 || r >= board.length || c < 0 || c >= board[0].length)
return;
        char ch = board[r][c];
        if (ch == '#' || !node.children.containsKey(ch)) return;

        node = node.children.get(ch);
        path.append(ch);

        if (node.isEnd) {
            result.add(path.toString());
            node.isEnd = false;   // avoid duplicates
        }

        board[r][c] = '#';
        dfs(board, r + 1, c, node, path);
        dfs(board, r - 1, c, node, path);
        dfs(board, r, c + 1, node, path);
        dfs(board, r, c - 1, node, path);
        board[r][c] = ch;
        path.deleteCharAt(path.length() - 1);
    }
}
```

**Why the trie is game-changing here:** Without a trie, you'd search the board once for each word — hugely wasteful because words share prefixes. With a trie, all shared prefixes are explored ONCE. The improvement can be 100× or more when you have many words sharing prefixes.

## 11. Classic Problem 4 — Longest Word in Dictionary

Given a list of words, find the LONGEST word that can be built one character at a time from other words in the list. Insert all words into a trie, then DFS — only descend into children where isEnd is true.

```
class Solution {
    String best = "";

    public String longestWord(String[] words) {
        Trie trie = new Trie();
        for (String w : words) trie.insert(w);
        dfs(trie.root, new StringBuilder());
        return best;
    }

    void dfs(TrieNode node, StringBuilder path) {
        if (path.length() > best.length() ||
            (path.length() == best.length() &&
path.toString().compareTo(best) < 0)) {
```

```
            best = path.toString();
        }

        for (var entry : node.children.entrySet()) {
            TrieNode child = entry.getValue();
            if (child.isEnd) {                // only descend if valid
word
                path.append(entry.getKey());
                dfs(child, path);
                path.deleteCharAt(path.length() - 1);
            }
        }
    }
}
```

## 12. Classic Problem 5 — Design Add and Search Word (with Wildcards)

Support add(word) and search(word) where search can contain a '.' matching any letter. E.g., search("b.d") matches "bad", "bcd", etc.

```
class WordDictionary {
    TrieNode root = new TrieNode();

    public void addWord(String word) {
        TrieNode curr = root;
        for (char c : word.toCharArray()) {
            curr.children.putIfAbsent(c, new TrieNode());
            curr = curr.children.get(c);
        }
        curr.isEnd = true;
    }

    public boolean search(String word) {
        return searchHelper(word, 0, root);
    }

    private boolean searchHelper(String word, int i, TrieNode node) {
        if (i == word.length()) return node.isEnd;

        char c = word.charAt(i);
        if (c == '.') {
            for (TrieNode child : node.children.values()) {
                if (searchHelper(word, i + 1, child)) return true;
            }
            return false;
        }
```

```
        TrieNode child = node.children.get(c);
        if (child == null) return false;
        return searchHelper(word, i + 1, child);
    }
}
```

The wildcard '.' forces a branching search — try EVERY child. That's why it's O(alphabet ^ dots) worst case, but usually fast in practice.

## 13. When to Use a Trie

| If you need to... | Use |
|---|---|
| Autocomplete / suggestions | **Trie + DFS from prefix node** |
| Spellcheck / dictionary lookup | **Trie with isEnd flag** |
| Longest common prefix of many words | **Insert all → find deepest single-child chain** |
| Word search on a grid with dictionary | **Trie + DFS pruning by trie edges** |
| Auto-suggest based on prefix + rank | **Trie + heap at each node (top-K)** |
| IP routing (longest matching prefix) | **Trie of binary bits** |
| Contact search / T9 keyboard | **Trie of contact names** |

If prefix matching is the core operation, a trie is the answer. If you just need exact lookup, HashSet or HashMap is simpler.

## 14. Common Mistakes

### Mistake 1: Confusing search with startsWith

```
// search("cat") in a trie containing "cats":
// Walking through c-a-t reaches node.isEnd == false → return false ✓

// If your search doesn't check isEnd, it wrongly returns true.
return node != null;              // ✗ that's startsWith, not search
return node != null && node.isEnd; // ✓
```

### Mistake 2: Forgetting to mark isEnd

```
for (char c : word.toCharArray()) {
    curr.children.putIfAbsent(c, new TrieNode());
    curr = curr.children.get(c);
}
// ✗ Missing: curr.isEnd = true;
// Now search(word) always returns false!
```

### Mistake 3: Using int[26] for mixed-case input

int[26] only handles lowercase a-z. If the input can have uppercase, digits, or special chars, you need HashMap or a bigger array indexed properly.

### Mistake 4: Building a new trie for each word in Word Search II

The whole point is to insert ALL words once, then DFS the board. Rebuilding for each word defeats the purpose.

### Mistake 5: Forgetting to unmark board cells after DFS

Same as regular backtracking — mark, recurse, UN-mark. Otherwise cells get stuck as visited.

### Mistake 6: Deleting nodes carelessly

A node might be part of another word's path. Never delete a node just because you removed a word ending there — check children.isEmpty() and !isEnd first.

### Mistake 7: Storing strings inside nodes

Beginners often store the full word at each node. That defeats the space-sharing benefit. The word is IMPLICIT — you build it up during traversal by concatenating characters.

## 15. Best Practices

- Use int[26] for lowercase-only problems — noticeably faster than HashMap
- Always check BOTH "node exists" AND "isEnd is true" for exact word search
- For prefix questions, don't check isEnd — just check that navigation succeeded
- For Word Search II, insert all words first, then DFS the grid once
- When DFS-ing, mark isEnd=false after finding a word to avoid duplicates in results
- Use StringBuilder for building word paths during DFS — avoid string concatenation in loops
- For very large dictionaries, consider compressed tries (Radix / Patricia) for better memory

## 16. Quick Reference

| Concept | Meaning |
|---|---|
| **`Trie / Prefix Tree`** | Tree where each edge represents a character |
| **`TrieNode`** | Node with children map and isEnd flag |
| **`Root`** | Empty top node — no character |
| **`isEnd`** | Flag marking end of a valid inserted word |
| **`insert(word)`** | Walk down, creating missing nodes; set isEnd at last |

| Concept | Meaning |
|---|---|
| `search(word)` | Walk; return isEnd of final node (both must exist) |
| `startsWith(prefix)` | Walk; return true if all characters have a path |
| `Autocomplete` | Navigate to prefix + DFS collecting isEnd nodes |
| `Word Search II` | Trie + DFS on grid, prune by trie edges |
| `int[26] children` | Fast lookup for lowercase-only inputs |
| `HashMap children` | Flexible — handles any character |
| `L` | Length of the word (per-operation complexity) |

## 17. Practice Problems

Try these in order. The first four are must-know for interviews.

### Foundational

1. Implement Trie — insert, search, startsWith (LeetCode 208).
2. Design Add and Search Words Data Structure — with '.' wildcard (LeetCode 211).
3. Word Search II — find all dictionary words on a 2D grid (LeetCode 212).
4. Longest Word in Dictionary — buildable one char at a time (LeetCode 720).

### Autocomplete & Suggestions

5. Search Suggestions System — for each prefix of a search word, return top 3 matches (LeetCode 1268).
6. Auto-complete: given a prefix, return the top K most-searched completions.
7. Return all words in dictionary starting with a given prefix.

### Advanced Trie

8. Replace Words — replace each word in a sentence with its shortest root from a dictionary (LeetCode 648).
9. Concatenated Words — find all words that are concatenations of at least two other words (LeetCode 472).
10. Maximum XOR of Two Numbers in an Array — bit trie (LeetCode 421).
11. Palindrome Pairs — find all pairs whose concatenation is a palindrome (LeetCode 336).

### Design

12. Design a Search Autocomplete System (LeetCode 642).
13. Design a File System Path Handler using a trie.

## 18. Final Takeaway

Tries are a small, focused topic — but they turn expensive O(n × L) prefix operations into O(L). Once you've solved a few trie problems, they start feeling almost mechanical: insert, walk, mark, done.

For interviews, master the three core operations (insert, search, startsWith), the autocomplete pattern (navigate + DFS), and the Word Search II combo (trie + grid DFS). Those cover the vast majority of trie questions you'll see.

> **Key Takeaway:** A trie stores strings by their characters, sharing prefixes to save space and enable O(L) lookups. Each node has children and an isEnd flag. Master the three core operations — insert, search, startsWith — and the two big patterns: autocomplete via DFS from a prefix node, and Word Search II via trie + grid DFS. Choose int[26] for lowercase-only problems, HashMap for general use.

## 19. What's Next?

With tries in place, the remaining specialized data structures are:

- Union-Find (DSU) — fast connectivity and grouping problems
- Segment Tree / Fenwick Tree — range queries and updates
- Monotonic Stack / Deque — next greater element, largest rectangle
- Bit Manipulation — bitmask DP, XOR tricks, subset generation
- KMP / Rabin-Karp — advanced string pattern matching
- Advanced Graph — Minimum Spanning Tree (Kruskal, Prim), Bellman-Ford
- System Design — for later interview rounds